NAAN MUDHALVAN SMART BRIDGE

EDUGENIE - GOOGLE GEMINI POWERED LEARNING ASSISTANT

STUDENTNAME	**: Aishwarya, Aishwarya G, Magila G**
YEAR	**: III**
COLLEGE CODE	**: BRU5X**
COLLEGE NAME	**: ST.PAUL'S COLLEGE OF ARTS AND SCIENCE FOR WOMEN**
DEPARTMENT	**: COMPUTER SCIENCE**
SEMESTER	**: III**
ROLL NO	**: 2522K1186, 2522K1185, 2522K1190**
NMID	**: EE40158SF3D6421ADB24955DAF474EBC, 7CEAE6A0723118198514F79A27207FF3, 9E461D5432B56CF8D8A41096AB3D796B**
DATE	**: 10/10/2026**
TITLE	**: EDUGENIE - GOOGLE GEMINI POWERED LEARNING ASSISTANT**
GITHUB LINK	**: https://github.com/aishuais018-ai/Aishwarya-**

1. Project Overview

EduGenie is a student-focused web application that generates personalized study notes and a short practice quiz from the details a learner provides, such as name, education level, topic, and preferred learning style. It also supports feedback-based note updates (for example, "explain in simpler words"). A separate view allows administrators or developers to inspect stored learner records, original notes, and updated notes.

2. Problem Statement

Many students struggle to find study material that matches their level and the way they learn. Searching through textbooks and websites is time-consuming, and the content is rarely personalized. EduGenie addresses this by combining a simple web form, generative AI, and persistent storage to produce structured notes and a quiz, and to revise the notes based on student feedback.

3. Objectives

- Generate personalized study notes for any topic.

- Provide a short practice quiz for self-assessment.
- Accept feedback and revise existing notes.
- Store learner information and note history.
- Provide a clear browser-based interface.
- Demonstrate FastAPI, Jinja2, Gemini integration, SQLite, and SQLAlchemy in one project.

4. Project Scenarios

- Scenario 1: A student enters name, level, topic, and learning style and requests study notes.
- Scenario 2: A student provides feedback such as "use simpler words" or "add more examples", and the application generates revised notes.
- Scenario 3: The application generates a 5-question practice quiz for the selected topic.
- Scenario 4: An administrator/developer opens the all-users page to inspect learner details and original/updated notes.

5. Technical Architecture

The application follows a modular architecture: HTML/Jinja2 templates form the presentation layer; FastAPI handles routes and request processing; Gemini modules provide AI-generated content; SQLite with SQLAlchemy stores learner and notes information.

6. Project Workflow / Milestones

Milestone 1 – Model selection and architecture

- Understand AI requirements
- Define frontend, backend, AI and database layers
- Prepare the development environment

Milestone 2 – Core functionality

- Study-notes generation
- Practice-quiz generation
- Feedback-based notes updating
- User and notes persistence

Milestone 3 – routes.py development

- Home route
- Notes generation route
- Feedback route
- Admin/all-users route

Milestone 4 – Frontend development

- index.html
- result.html
- all_users.html
- Responsive and readable form/result presentation

Milestone 5 – Deployment and verification

- Install dependencies
- Configure environment variables
- Run with Uvicorn
- Verify browser pages and FastAPI /docs

7. Environment Setup

Create and activate a virtual environment, install requirements, configure GOOGLE_API_KEY, and run the FastAPI application with Uvicorn. The project contains a demo fallback so the interface can be demonstrated without an API key; live Gemini generation requires a valid configured API key.

8. Project Structure

```
EduGenie_Project/
|-- app/
|   |-- __init__.py
|   |-- main.py
|   |-- routes.py
|   |-- database.py
|   |-- gemini_notes.py
|   |-- gemini_quiz.py
|   `-- updated_notes.py
|-- templates/
|   |-- index.html
|   |-- result.html
|   `-- all_users.html
|-- static/
|   |-- output_home.png
|   |-- output_result.png
|   `-- output_users.png
|-- requirements.txt
|-- .env.example
|-- .gitignore
`-- README.md
```

9. Core Modules

- app/main.py – creates the FastAPI application and registers routes.
- app/routes.py – connects browser forms to AI and database operations.
- app/database.py – defines the SQLAlchemy model and persistence helpers.
- app/gemini_notes.py – generates personalized study notes (Gemini Pro).
- app/gemini_quiz.py – generates the practice quiz (Gemini Flash).
- app/updated_notes.py – revises the notes from student feedback.
- templates/index.html – input page.
- templates/result.html – notes, quiz, and feedback page.
- templates/all_users.html – stored-user/admin view.

10. Functional Flow

1. User opens the home page.
2. User enters name, ID, level, topic, and learning style.
3. FastAPI receives the form.
4. Notes-generation and quiz-generation modules are called.
5. Learner information and notes are stored in SQLite.
6. result.html displays the personalized output.

7. User may submit feedback.
8. The original notes plus feedback are passed to the update module.
9. The revised notes are stored and displayed.
10. The all-users page displays stored records.

11. Frontend Pages

- Home page: collects name, user ID, education level, topic, and learning style.
- Result page: shows learner details, study notes, practice quiz, and feedback form.
- All-users page: displays saved records and original/updated notes.

12. Database Design

The User model stores user_id, username, level, topic, style, original_notes, updated_notes, and quiz. The database layer provides functions for saving users/notes, retrieving a user, updating notes, and retrieving all users.

13. Testing Plan

Test valid form submission, missing fields, repeated user IDs, feedback submission, unknown user IDs, database persistence, demo fallback behavior, and page rendering. For live AI testing, verify that GOOGLE_API_KEY is configured and that the Gemini service is reachable.

14. Output / Demonstration

The package includes demonstration interface screenshots (static/output_home.png, output_result.png, output_users.png). These screenshots show the demo-fallback output and should not be described as live Gemini responses unless the application is run with a valid API key.

15. Deployment

Windows setup:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Then open http://127.0.0.1:8000 and the FastAPI documentation at http://127.0.0.1:8000/docs.

16. Installation in VS Code

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
copy .env.example .env
uvicorn app.main:app --reload
```

Then open http://127.0.0.1:8000

17. Security and Responsible Use

Keep API credentials in environment variables and never commit real keys. AI-generated study material can contain mistakes, so students should verify important facts with textbooks or teachers. EduGenie is a learning aid and not a replacement for formal instruction.

18. Limitations

The project is a demonstration application. AI output can vary, the demo fallback is not a live AI response, and there is no full authentication/authorization layer in this local package.

19. Future Enhancements

Login and role-based access, quiz scoring and progress charts, PDF export of notes, multi-language (Tamil/English) notes, document upload for summarization, stronger validation, cloud database support, production logging, and automated tests.

20. Conclusion

EduGenie demonstrates an end-to-end AI web application in which a student submits a topic, receives structured study notes and a practice quiz, provides feedback, and gets revised notes. The implementation combines FastAPI, Jinja2, Gemini integration, SQLite, and SQLAlchemy in a modular project structure.

21. Complete Code Files

Actual source files included in the final project package are reproduced below for documentation and submission.

1. app/main.py

```
from fastapi import FastAPI
from .routes import router

app = FastAPI(
    title="EduGenie - Gemini Powered Learning Assistant",
    description="AI-powered personalized study notes and quizzes using Gemini models.",
    version="1.0.0",
)
app.include_router(router)

@app.get("/health")
def health():
    return {"status": "ok", "application": "EduGenie"}
```

2. app/routes.py

```
from fastapi import APIRouter, Request, Form
from fastapi.responses import HTMLResponse
from fastapi.templating import Jinja2Templates
from .gemini_notes import generate_study_notes
from .gemini_quiz import generate_quiz
from .updated_notes import update_study_notes
from .database import save_user, save_content, get_user, get_all_users, update_notes

router = APIRouter()
templates = Jinja2Templates(directory="templates")

@router.get("/", response_class=HTMLResponse)
def home(request: Request):
    return templates.TemplateResponse("index.html", {"request": request})

@router.post("/generate-notes", response_class=HTMLResponse)
def generate_notes(request: Request, username: str = Form(...),
                   user_id: str = Form(...), level: str = Form(...),
                   topic: str = Form(...), style: str = Form(...)):
    notes = generate_study_notes(username, level, topic, style)
    quiz = generate_quiz(topic, level)
    if get_user(user_id):
        save_content(user_id, topic, notes, quiz)
    else:
        save_user({"user_id": user_id, "username": username, "level": level,
                   "topic": topic, "style": style, "original_notes": notes,
                   "updated_notes": None, "quiz": quiz})
    return templates.TemplateResponse("result.html", {
        "request": request, "username": username, "user_id": user_id,
        "level": level, "topic": topic, "style": style,
        "notes": notes, "quiz": quiz, "updated": False})
```

```
@router.post("/submit-feedback", response_class=HTMLResponse)
def submit_feedback(request: Request, user_id: str = Form(...),
                    feedback: str = Form(...)):
    user = get_user(user_id)
    if not user:
        return templates.TemplateResponse("result.html", {
            "request": request,
            "error": "User ID not found. Please generate notes first."})
    revised = update_study_notes(user.original_notes, feedback)
    update_notes(user_id, revised)
    return templates.TemplateResponse("result.html", {
        "request": request, "username": user.username, "user_id": user.user_id,
        "level": user.level, "topic": user.topic, "style": user.style,
        "notes": revised, "quiz": user.quiz, "updated": True})

@router.get("/view-all-users", response_class=HTMLResponse)
def view_all_users(request: Request):
    return templates.TemplateResponse("all_users.html", {
        "request": request, "users": get_all_users()})
```

3. app/database.py

```
from sqlalchemy import create_engine, Column, Integer, String, Text
from sqlalchemy.orm import declarative_base, sessionmaker

DATABASE_URL = "sqlite:///./edugenie.db"
engine = create_engine(DATABASE_URL, connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()

class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True, index=True)
    user_id = Column(String(50), unique=True, index=True, nullable=False)
    username = Column(String(100), nullable=False)
    level = Column(String(50), nullable=False)
    topic = Column(String(200), nullable=False)
    style = Column(String(50), nullable=False)
    original_notes = Column(Text, nullable=False)
    updated_notes = Column(Text, nullable=True)
    quiz = Column(Text, nullable=True)

Base.metadata.create_all(bind=engine)

def save_user(data):
    db = SessionLocal()
    try:
        user = User(**data)
        db.add(user)
        db.commit()
        db.refresh(user)
        return user
    finally:
        db.close()

def save_content(user_id, topic, notes, quiz=None):
    db = SessionLocal()
    try:
        user = db.query(User).filter(User.user_id == user_id).first()
        if user:
            user.topic = topic
            user.original_notes = notes
            user.updated_notes = None
            user.quiz = quiz
            db.commit()
    finally:
        db.close()

def update_notes(user_id, updated_notes):
    db = SessionLocal()
    try:
        user = db.query(User).filter(User.user_id == user_id).first()
        if user:
            user.updated_notes = updated_notes
            db.commit()
        return user
    finally:
```

```
        db.close()

def get_user(user_id):
    db = SessionLocal()
    try:
        return db.query(User).filter(User.user_id == user_id).first()
    finally:
        db.close()

def get_all_users():
    db = SessionLocal()
    try:
        return db.query(User).order_by(User.id.desc()).all()
    finally:
        db.close()
```

4. app/gemini_notes.py

```
import os
try:
    from google import genai
except ImportError:
    genai = None

def _demo_notes(topic, level, style):
    return f"""STUDY NOTES: {topic.title()}
Level: {level.title()} | Learning style: {style.title()}

1. Introduction
   A short overview of {topic} and why it matters.
2. Key Concepts
   - Concept A: simple definition
   - Concept B: simple definition
   - Concept C: simple definition
3. Example
   A worked example that shows how {topic} is used.
4. Quick Revision
   Three points to remember before an exam.
Note: This is demo content, not a live Gemini response."""

def generate_study_notes(name, level, topic, style):
    api_key = os.getenv("GOOGLE_API_KEY")
    if not api_key or genai is None:
        return _demo_notes(topic, level, style)
    client = genai.Client(api_key=api_key)
    prompt = f"""Create clear study notes for a student.
Student name: {name}
Education level: {level}
Topic: {topic}
Preferred learning style: {style}
Include: introduction, key concepts, one worked example, common mistakes,
and a short revision summary. Keep it accurate and easy to follow."""
    response = client.models.generate_content(
        model="gemini-2.5-pro", contents=prompt)
    return response.text
```

5. app/gemini_quiz.py

```
import os
try:
    from google import genai
except ImportError:
    genai = None

def generate_quiz(topic, level):
    api_key = os.getenv("GOOGLE_API_KEY")
    if not api_key or genai is None:
        return (f"Practice quiz on {topic}:\n"
                "Q1. Define the main idea of the topic.\n"
                "Q2. Give one real-life example.\n"
                "Q3. List two key terms and explain them.")
    client = genai.Client(api_key=api_key)
    prompt = (f"Write 5 multiple-choice questions on '{topic}' for a "
              f"{level} student. Give four options each and mark the "
              "correct answer at the end of every question.")
    return client.models.generate_content(
        model="gemini-2.5-flash", contents=prompt).text
```

6. app/updated_notes.py

```
import os
try:
    from google import genai
except ImportError:
    genai = None

def update_study_notes(original_notes, feedback):
    api_key = os.getenv("GOOGLE_API_KEY")
    if not api_key or genai is None:
        return (original_notes + f"\n\nUPDATED USING FEEDBACK:\n{feedback}\n\n"
                "The revised notes should follow the requested preference.")
    client = genai.Client(api_key=api_key)
    prompt = f"""Original study notes:
{original_notes}

Student feedback:
{feedback}

Rewrite the notes to follow the feedback (for example simpler words, more
examples, or shorter points) while keeping the structure clear."""
    return client.models.generate_content(
        model="gemini-2.5-pro", contents=prompt).text
```

7. templates/index.html

```
<!doctype html>
<html>
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>EduGenie - Learning Assistant</title>
<style>
body{font-family:Arial,sans-serif;margin:0;background:linear-gradient(135deg,#1e1b4b,#312e81);c
    olor:#fff;min-height:100vh}
.card{max-width:720px;margin:45px auto;padding:32px;background:#fff;color:#111827;border-radius
    :18px;box-shadow:0 15px 40px #0008}
h1{margin-top:0;color:#4338ca}
.grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
label{font-weight:700}
input,select{width:100%;padding:11px;margin-top:6px;border:1px solid #cbd5e1;border-radius:9px;
    box-sizing:border-box}
.full{grid-column:1/-1}
button{background:#4338ca;color:#fff;border:0;padding:13px 22px;border-radius:10px;font-weight:
    700;cursor:pointer}
@media(max-width:650px){.grid{grid-template-columns:1fr}.full{grid-column:auto}}
</style>
</head>
<body><div class="card">
<h1>EduGenie</h1><p>Google Gemini Powered Learning Assistant</p>
<form method="post" action="/generate-notes"><div class="grid">
<div><label>Name</label><input name="username" required></div>
<div><label>User ID</label><input name="user_id" required></div>
<div><label>Education Level</label><select name="level"><option>School</option><option>Undergra
    duate</option><option>Postgraduate</option></select></div>
<div><label>Learning Style</label><select name="style"><option>Simple</option><option>Example-b
    ased</option><option>Detailed</option></select></div>
<div class="full"><label>Topic</label><input name="topic" placeholder="Example: Photosynthesis"
     required></div>
<div class="full"><button type="submit">Generate Notes</button></div>
</div></form></div></body></html>
```

8. templates/result.html

```
<!doctype html><html><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>EduGenie Result</title>
<style>
body{font-family:Arial;background:#f1f5f9;color:#0f172a;margin:0}
.wrap{max-width:1000px;margin:30px auto;padding:20px}
.card{background:#fff;border-radius:16px;padding:22px;margin-bottom:20px;box-shadow:0 5px 20px
    #0001}
pre{white-space:pre-wrap;background:#0f172a;color:#e2e8f0;padding:18px;border-radius:12px;line-
    height:1.5}
textarea{width:100%;padding:12px;box-sizing:border-box;border:1px solid #cbd5e1;border-radius:9
    px}
```

```
button{background:#4338ca;color:#fff;border:0;padding:12px 20px;border-radius:9px;font-weight:7
    00}
.badge{display:inline-block;padding:6px 10px;border-radius:20px;background:#e0e7ff;color:#3730a
    3}
</style></head><body><div class="wrap">
{% if error %}<div class="card"><p>{{ error }}</p><a href="/">Go back</a></div>
{% else %}
<div class="card"><h1>Your Study Notes</h1>
<p><b>Name:</b> {{ username }}   <b>User ID:</b> {{ user_id }}</p>
<p><b>Level:</b> {{ level }}   <b>Style:</b> {{ style }}</p>
<p><b>Topic:</b> <span class="badge">{{ topic }}</span></p>
{% if updated %}<p class="badge">Notes updated using your feedback.</p>{% endif %}</div>
<div class="card"><h2>Study Notes</h2><pre>{{ notes }}</pre></div>
<div class="card"><h2>Practice Quiz</h2><pre>{{ quiz }}</pre></div>
<div class="card"><h2>Improve Your Notes</h2>
<form method="post" action="/submit-feedback">
<input type="hidden" name="user_id" value="{{ user_id }}">
<textarea name="feedback" rows="4" placeholder="Example: Explain in simpler words with more exa
    mples." required></textarea>
<br><br><button type="submit">Update Notes</button></form></div>
<p><a href="/">New topic</a> | <a href="/view-all-users">View all users</a></p>
{% endif %}
</div></body></html>
```

9. templates/all_users.html

```
<!doctype html><html><head><meta charset="utf-8"><title>EduGenie Admin</title>
<style>
body{font-family:Arial;background:#f8fafc;margin:0}
.wrap{padding:25px}
table{width:100%;border-collapse:collapse;background:#fff}
th,td{border:1px solid #cbd5e1;padding:10px;vertical-align:top}
th{background:#4338ca;color:#fff}
pre{white-space:pre-wrap;max-width:420px}
</style></head><body><div class="wrap"><h1>EduGenie - View All Users</h1>
<table><tr><th>User ID</th><th>Name</th><th>Level</th><th>Topic</th><th>Style</th><th>Original
    Notes</th><th>Updated Notes</th></tr>
{% for user in users %}<tr><td>{{ user.user_id }}</td><td>{{ user.username }}</td><td>{{ user.l
    evel }}</td><td>{{ user.topic }}</td><td>{{ user.style }}</td><td><pre>{{ user.original_not
    es }}</pre></td><td><pre>{{ user.updated_notes or "Not updated" }}</pre></td></tr>{% endfor
     %}
</table></div></body></html>
```

10. requirements.txt

```
fastapi
uvicorn
jinja2
sqlalchemy
python-multipart
google-genai
python-dotenv
```

11. .env.example

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

README.md

```
# EduGenie - Google Gemini Powered Learning Assistant
## Features
- Learner input form
- AI-generated personalized study notes (Gemini Pro)
- Practice quiz generation (Gemini Flash)
- Feedback-based notes updating
- SQLite + SQLAlchemy persistence, Jinja2 frontend
- Admin view of users and original/updated notes
- FastAPI /docs interactive API documentation
- Demo fallback output when no Gemini API key is configured
## Run
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env   (add GOOGLE_API_KEY)
uvicorn app.main:app --reload
Open http://127.0.0.1:8000 and http://127.0.0.1:8000/docs
```

SAMPLE OUTPUT / DEMONSTRATION

```
Input:
Name: Anu   User ID: ED001   Level: Undergraduate
Topic: Photosynthesis   Learning style: Simple

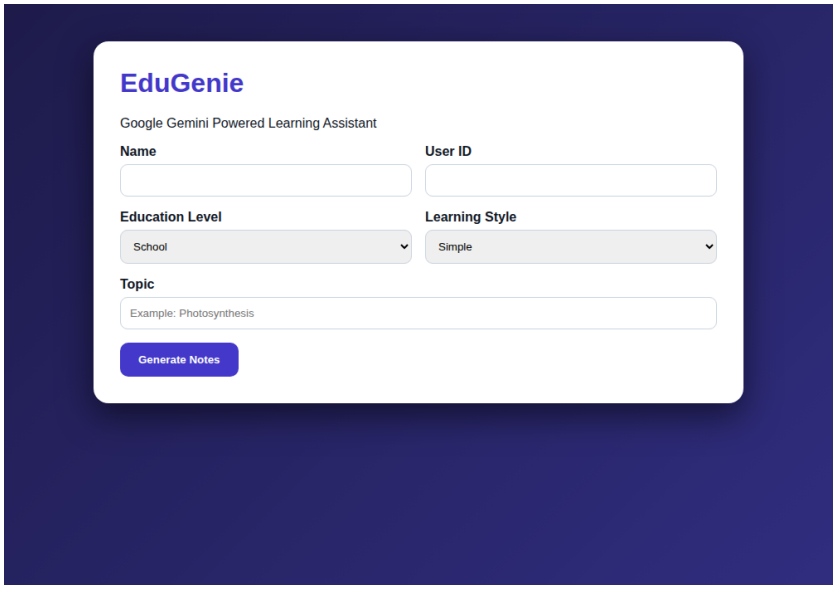
Output:
STUDY NOTES: Photosynthesis
1. Introduction
2. Key Concepts
3. Example
4. Quick Revision

Practice Quiz:
Q1. Define the main idea of the topic.
Q2. Give one real-life example.
Q3. List two key terms and explain them.

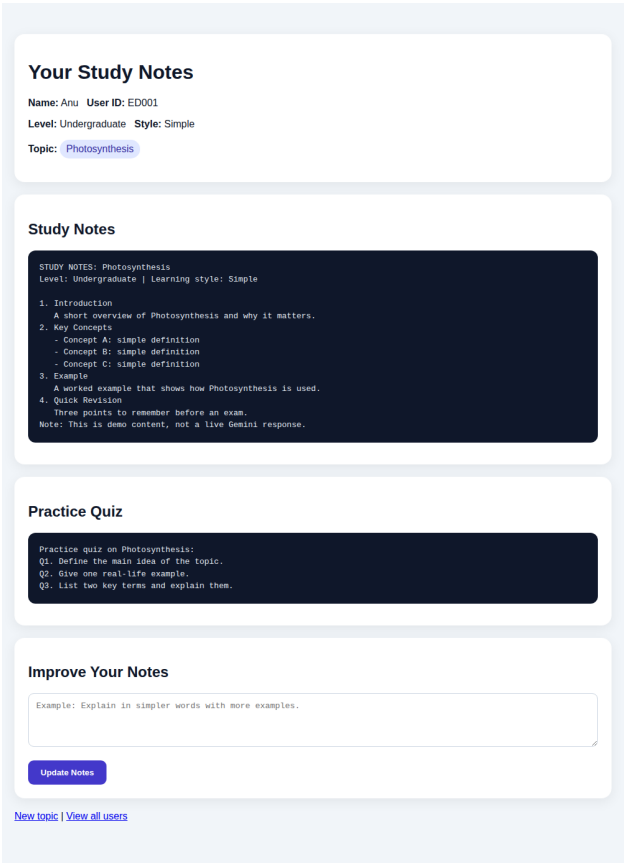
Feedback: "Explain in simpler words with more examples."
Updated Output: Notes revised to follow the feedback.
```

22. Demonstration Output Screens

Output 1 – Home / input page



Output 2 – Generated Study Notes and Quiz



Output 3 – Admin / All Users Page

